Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
Département Ingénierie des Systèmes Embarqués
Filière Smart Systems Engineering

TP 2 : Détection, suivi et classification à règles sur le tapis de la station EcoTri 4.0

Projet fédérateur EcoTri 4.0, sous-projet SP2 et jalon J2

Pr.Hassan BERBIA/Pr.Aissam BERRAHOU

31 octobre 2026

Objectifs du TP

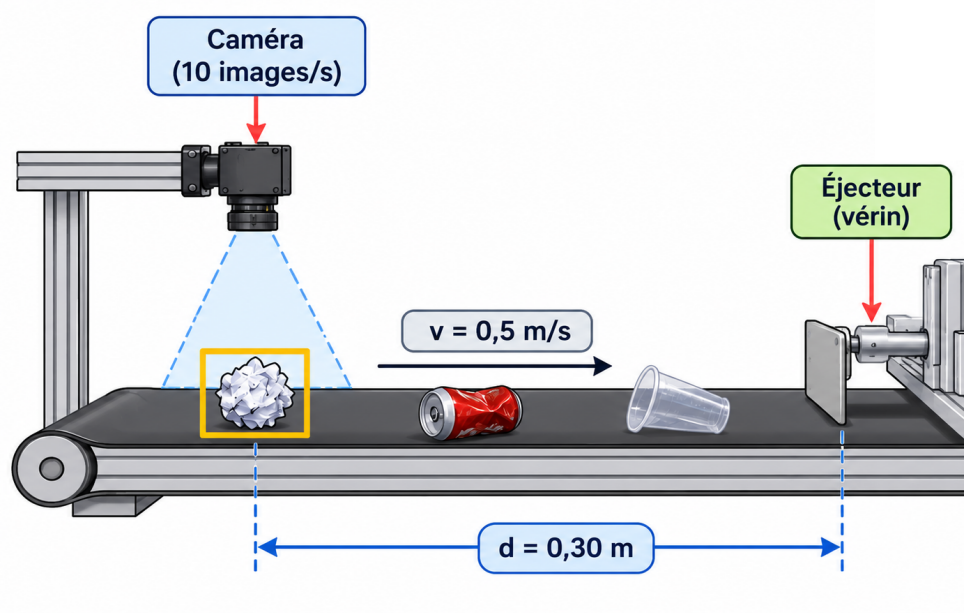
Le Tutorial 2 a montré chaque opérateur isolément. Dans ce TP, chaque équipe (3 ou 4 étudiants) les assemble en un système qui **trie** : la Portenta détecte, suit et classe les objets qui défilent ; l'ESP32-CAM identifie le lot et diffuse la vue d'ensemble. Le chiffre le plus important du TP est le **taux d'erreur** de votre classifieur : il servira de référence au réseau de neurones de la lecture 5.
À la fin de ce TP, vous serez capable de :

- Calibrer une scène de vision (ROI, exposition, gain, échelle, mire de référence).
- Détecter et suivre des objets en mouvement, les compter et émettre un message JSON par objet.
- Concevoir un classifieur à règles à partir de mesures et l'évaluer par une matrice de confusion.
- Lire le QR code d'un lot et diffuser un flux vidéo Wi-Fi avec l'ESP32-CAM, en mesurant leurs cadences.

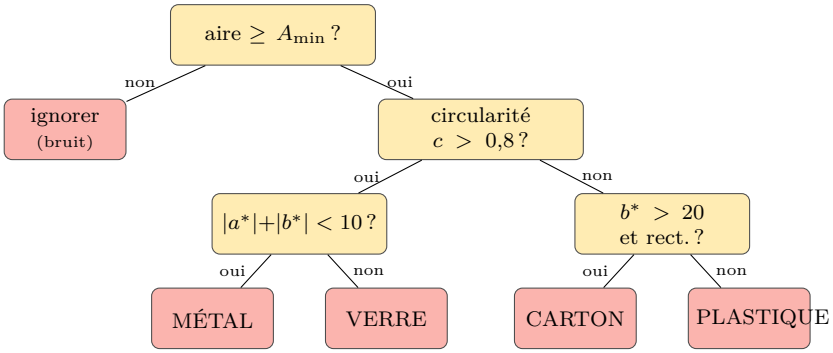
Livrable du jalon J2 (à déposer en fin de lecture) : scripts, arbre de règles, matrice de confusion et mesures (partie 6).

1 Contexte : ce que la station doit faire

Les objets défilent sur le tapis. La Portenta, au-dessus de la zone de décision, doit attribuer à chacun une classe parmi **métal, carton, plastique, verre** ; la caméra de supervision lit l'étiquette QR du lot qui arrive sur le tapis.



La zone traitée dans ce TP : la caméra de décision au-dessus du tapis et l'éjecteur 30 cm plus loin. Chaque objet doit être classé une seule fois, avant d'atteindre l'éjecteur.



Le type de classifieur que vous allez construire : un arbre de règles. Sur la Portenta (monochrome), les tests de couleur sont remplacés par des descripteurs de luminance et de reflets ; à vous de choisir les seuils à partir de vos mesures.

2 Pré-requis

Matériel (par équipe)

- **Portenta H7 + Vision Shield** fixée 25–35 cm au-dessus d'un fond sombre (maquette du tapis ou bande de carton noir que l'on fait glisser).
- **ESP32-CAM AI-Thinker** + adaptateur ESP32-CAM-MB, un point d'accès Wi-Fi (partage de connexion d'un smartphone).

- **20 objets par classe** au minimum (canettes, cartons, bouteilles, bocaux ou flacons), dont certains abîmés : écrasés, pliés, étiquetés.
- Une lampe de bureau, une feuille grise ou blanche (mire de référence), un QR code imprimé contenant `{"lot":"L-2027-0412"}`.

Logiciel

1. **OpenMV IDE** (Tutorial 2 terminé, tableau de l'étape 4 rempli).
2. **Arduino IDE 2.x** avec le paquet **esp32 by Espressif Systems** (TP1) et la bibliothèque **ESP32QRCodeReader** (gestionnaire de bibliothèques, ou github.com/alvarowolfx/ESP32QRCodeReader).
3. **Python 3** sur le PC avec `numpy`, `opencv-python` et `matplotlib` pour l'analyse.

3 Étapes détaillées du TP

Partie 1 : Calibrer la scène (15 min)

Avant de régler le moindre seuil, la scène doit être **reproductible**. Le script suivant fige les réglages du capteur, mesure la mire de référence et l'échelle, puis enregistre la configuration dans un fichier de la carte.

```
# TP2 - partie 1 : calibration de la scène (Portenta + Vision Shield)
import sensor, time, json

TAPIS = (0, 40, 320, 160)          # bande utile (x, y, w, h)
MIRE  = (4, 4, 24, 24)             # petit carré gris fixé dans un coin du champ
LARGEUR_REF_CM = 6.6               # diamètre réel de la canette de référence

sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QVGA)
sensor.skip_frames(time=2500)                       # tapis VIDE pendant la convergence
try:
    g, e = sensor.get_gain_db(), sensor.get_exposure_us()
    sensor.set_auto_gain(False, gain_db=g)
    sensor.set_auto_exposure(False, exposure_us=e)
except Exception:
    g, e = None, None

img = sensor.snapshot()
fond = img.get_statistics(roi=TAPIS)
mire = img.get_statistics(roi=MIRE)
seuil = img.get_histogram(roi=TAPIS).get_threshold().value()
print("Fond : moyenne %d, écart-type %.1f | mire %d | Otsu %d" % (fond.mean(), fond.
    stdev(), mire.mean(), seuil))

print("Posez la canette de référence au centre (5 s)...")
time.sleep_ms(5000)
img = sensor.snapshot()
objet = [(fond.mean() + 4 * int(fond.stdev()) + 10, 255)]   # plus clair que le fond
blobs = img.find_blobs(objet, roi=TAPIS, pixels_threshold=200)
b = max(blobs, key=lambda b: b.pixels()) if blobs else None
px_par_cm = b.w() / LARGEUR_REF_CM if b else None
print("Échelle : %s px/cm" % px_par_cm)
```

```
conf = {"gain_db": g, "expo_us": e, "fond": fond.mean(), "sd_fond": fond.stdev(),
        "mire": mire.mean(), "seuil_objet": objet[0][0], "px_par_cm": px_par_cm}
with open("/calib.json", "w") as f:
    f.write(json.dumps(conf))
print(conf)
```

1. Le seuil objet est fixé à 4 écarts-types au-dessus du fond : justifiez ce choix par la proportion de pixels du fond qui le dépasseraient si le bruit était gaussien.
2. Changez l'éclairage (lampe plus proche) **sans** recalibrer, puis relancez seulement la mesure de la mire. De combien a-t-elle varié ? Proposez une correction des seuils à partir de ce rapport.

Partie 2 : Détection, suivi et comptage des objets (35 min)

Un objet reste visible pendant plusieurs images : sans **suivi**, il serait compté et classé plusieurs fois. On associe chaque blob de l'image courante au blob le plus proche de l'image précédente (plus proche voisin), on lui donne un identifiant, et l'on déclenche la classification **une seule fois**, quand son centroïde franchit une ligne virtuelle.

```
# TP2 - partie 2 : détection + suivi + franchissement de ligne + JSON
import sensor, time, json, math
from pyb import LED

conf = json.loads(open("/calib.json").read())
TAPIS = (0, 40, 320, 160)
LIGNE_X = 160                       # le tapis défile de gauche à droite
OBJET = [(int(conf["seuil_objet"]), 255)]
D_MAX = 40                          # déplacement max entre 2 images (pixels)

sensor.reset(); sensor.set_pixformat(sensor.GRAYSCALE); sensor.set_framesize(sensor.
    QVGA)
sensor.skip_frames(time=500)
if conf["gain_db"] is not None:
    sensor.set_auto_gain(False, gain_db=conf["gain_db"])
    sensor.set_auto_exposure(False, exposure_us=int(conf["expo_us"]))

pistes = {}                         # id -> (cx, cy, déjà_compté)
prochain_id, compteur = 0, 0
clock = time.clock()

def descripteurs(img, b):
    st = img.get_statistics(roi=b.rect())
    hist = img.get_histogram(roi=b.rect()).bins()
    return {"A": b.pixels(), "rond": round(b.roundness(), 3), "elong": round(b.
    elongation(), 3),
            "dens": round(b.density(), 3), "L": st.mean(), "sd": round(st.stdev(), 1)
    ,
            "sat": round(sum(hist[-8:]), 3)}       # part de pixels quasi saturés (
    reflets)

def classer(d):
    return "inconnu"                               # complété en partie 3

while True:
    clock.tick()
    img = sensor.snapshot()
    t0 = time.ticks_us()
    blobs = img.find_blobs(OBJET, roi=TAPIS, pixels_threshold=400, area_threshold
    =400,
```

```
                            merge=True, margin=5)
    nouvelles = {}
    for b in blobs:
        # association au plus proche voisin
        meilleur, dmin = None, D_MAX
        for i, (x, y, c) in pistes.items():
            d = math.sqrt((b.cx() - x) ** 2 + (b.cy() - y) ** 2)
            if d < dmin and i not in nouvelles:
                meilleur, dmin = i, d
        if meilleur is None:
            meilleur, prochain_id = prochain_id, prochain_id + 1
            deja = False
        else:
            deja = pistes[meilleur][2]
        # franchissement de la ligne de décision
        if not deja and b.cx() >= LIGNE_X:
            d = descripteurs(img, b)
            msg = {"t": time.ticks_ms(), "id": meilleur, "cls": classer(d),
                   "cx": b.cx(), "cy": b.cy()}
            msg.update(d)
            print(json.dumps(msg))
            compteur += 1; deja = True
            LED(2).on(); LED(2).off()
        nouvelles[meilleur] = (b.cx(), b.cy(), deja)
        img.draw_rectangle(b.rect(), color=255)
        img.draw_string(b.x(), b.y() - 10, str(meilleur), color=255)
    pistes = nouvelles
    img.draw_line(LIGNE_X, TAPIS[1], LIGNE_X, TAPIS[1] + TAPIS[3], color=128)
    dt = time.ticks_diff(time.ticks_us(), t0) / 1000
    print("# %d objets comptés | traitement %.1f ms | %.1f FPS" % (compteur, dt,
    clock.fps()))
```

1. Faites défiler 10 objets à vitesse modérée : le compteur doit valoir exactement 10. Notez les erreurs (double comptage, objet manqué) et leur cause.
2. Augmentez la vitesse de défilement. À partir de quel déplacement entre deux images le suivi casse-t-il ? Reliez ce déplacement à `D_MAX`, à la cadence mesurée et à la vitesse du tapis ($v = 0{,}5$ m/s dans la station réelle) : la cadence de la Portenta est-elle suffisante ?
3. Que se passe-t-il quand deux objets se touchent ? Proposez une parade (règle sur l'aire maximale, espacement imposé sur le tapis).

Astuce

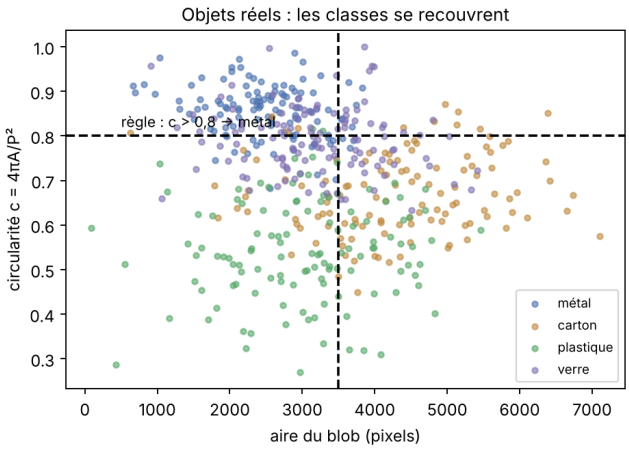
Les lignes qui commencent par `#` sont des messages de diagnostic ; les lignes JSON sont les **données**. Copiez le terminal dans un fichier texte : la partie 3 n'utilise que les lignes JSON.

Partie 3 : Collecte des descripteurs et conception des règles (30 min)

1. **Jeu de réglage.** Faites passer **10 objets par classe** (pas les mêmes que ceux de la partie 4), en notant la classe réelle de chaque identifiant. Enregistrez les lignes JSON dans `reglage.jsonl` et ajoutez à la main un champ `"vrai"` à chaque ligne.
2. **Visualisation.** Sur le PC, tracez les descripteurs deux à deux pour repérer ceux qui séparent les classes :

```
# analyse_descripteurs.py - à exécuter sur le PC
import json, itertools, matplotlib.pyplot as plt
```

```
data = [json.loads(l) for l in open("reglage.jsonl") if l.startswith("{")]
desc = ["A", "rond", "elong", "dens", "L", "sd", "sat"]
classes = sorted({d["vrai"] for d in data})
fig, axes = plt.subplots(3, 4, figsize=(14, 9))
for ax, (x, y) in zip(axes.ravel(), itertools.combinations(desc, 2)):
    for c in classes:
        pts = [(d[x], d[y]) for d in data if d["vrai"] == c]
        ax.scatter(*zip(*pts), s=14, label=c)
    ax.set_xlabel(x); ax.set_ylabel(y)
axes[0, 0].legend(); plt.tight_layout(); plt.savefig("descripteurs.png", dpi=120)
```



Ce que l'on obtient typiquement : les classes se recouvrent partiellement. Cherchez les couples de descripteurs qui les séparent le mieux et placez vos seuils dans les zones de faible recouvrement.

3. **Règles.** Complétez la fonction `classer` de la partie 2 avec vos seuils. Exemple de structure à adapter :

```
def classer(d):
    k = conf["px_par_cm"] ** 2 if conf["px_par_cm"] else 1.0
    aire_cm2 = d["A"] / k                          # aire indépendante de la hauteur camé
    ra
    if d["rond"] > 0.80:                           # objets ronds vus de dessus
        if d["sat"] > 0.05:                        # forts reflets spéculaires
            return "metal"
        return "verre"
    if d["dens"] > 0.85 and d["sd"] < 20:          # rectangle plein, texture mate
        return "carton"
    return "plastique"
```

Question

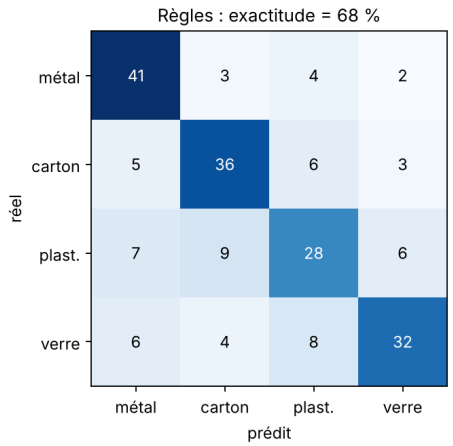
Pour chaque seuil, notez la **mesure** qui l'a motivé (valeur moyenne et dispersion par classe). Quelle classe vos descripteurs en niveaux de gris séparent-ils le plus mal ? Quel descripteur de couleur aiderait (voir la partie 5) ?

Partie 4 : Évaluation sur un jeu de test indépendant (20 min)

Faites passer **au moins 20 nouveaux objets par classe**, jamais vus pendant le réglage, avec vos règles actives. Enregistrez le résultat dans `test.jsonl` avec le champ `"vrai"`, puis calculez les

indicateurs :

```
# evaluation.py - matrice de confusion, exactitude, précision, rappel, intervalle de
    confiance
import json, math, numpy as np, matplotlib.pyplot as plt
data = [json.loads(l) for l in open("test.jsonl") if l.startswith("{")]
cls = ["metal", "carton", "plastique", "verre"]
C = np.zeros((4, 4), int)
for d in data:
    if d["cls"] in cls:
        C[cls.index(d["vrai"]), cls.index(d["cls"])] += 1
n = C.sum(); acc = np.trace(C) / n
ic = 1.96 * math.sqrt(acc * (1 - acc) / n)
print("Exactitude : %.1f %% (IC 95 %% : %.1f - %.1f), n = %d" % (100*acc, 100*(acc-ic
    ), 100*(acc+ic), n))
for k, c in enumerate(cls):
    P = C[k, k] / max(C[:, k].sum(), 1); R = C[k, k] / max(C[k, :].sum(), 1)
    print("%-10s précision %.2f  rappel %.2f  F1 %.2f" % (c, P, R, 2*P*R / max(P + R,
     1e-9)))
plt.imshow(C, cmap="Blues"); plt.xticks(range(4), cls); plt.yticks(range(4), cls)
for i in range(4):
    for j in range(4): plt.text(j, i, C[i, j], ha="center", va="center")
plt.xlabel("prédit"); plt.ylabel("réel"); plt.title("Classifieur à règles (J2)")
plt.savefig("confusion_J2.png", dpi=150)
```

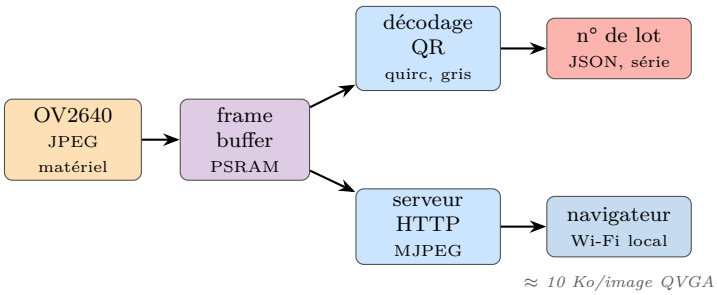


Exemple de résultat (200 objets). Votre matrice sera la référence du jalon J5.

Question

Comparez l'exactitude sur le jeu de réglage et sur le jeu de test. Laquelle est la plus élevée, et pourquoi ? Votre classifieur respecte-t-il l'exigence ENF-2 de votre cahier des charges (J1) ? Si l'écart est dans l'intervalle de confiance, que pouvez-vous conclure ?

Partie 5 : La caméra de supervision, QR du lot et flux Wi-Fi (30 min)



Les deux usages de l'ESP32-CAM. On les programme dans deux sketchs séparés et l'on mesure la cadence de chacun.

a) Lecture du QR du lot. La bibliothèque ESP32QRCodeReader configure la caméra en niveaux de gris et décode avec **quirc** dans une tâche FreeRTOS dédiée.

```
// TP2 - partie 5a : lecture du QR du lot (ESP32-CAM AI-Thinker)
#include <Arduino.h>
#include <ESP32QRCodeReader.h>

ESP32QRCodeReader reader(CAMERA_MODEL_AI_THINKER);
uint32_t nLus = 0, t0;

void tacheQR(void *pv) {
  struct QRCodeData qr;
  while (true) {
    if (reader.receiveQrCode(&qr, 100)) {
      if (qr.valid) {
        nLus++;
        Serial.printf("{\"t\":%lu,\"qr\":%s,\"lectures_s\":%.2f}\n",
                      millis(), (const char *)qr.payload, nLus * 1000.0 / (millis() -
    t0));
      } else {
        Serial.printf("# QR détecté mais illisible : %s\n", (const char *)qr.payload)
   ;
      }
    }
    vTaskDelay(50 / portTICK_PERIOD_MS);
  }
}

void setup() {
  Serial.begin(115200);
  reader.setup();
  reader.beginOnCore(1);                    // décodage sur le coeur 1
  t0 = millis();
  xTaskCreate(tacheQR, "tacheQR", 4 * 1024, NULL, 4, NULL);
}

void loop() { delay(100); }
```

1. Présentez l'étiquette à différentes distances : notez la distance maximale de lecture et calculez le nombre de pixels par module correspondant (la bibliothèque travaille en QVGA, 320×240).
2. Mesurez le nombre de lectures par seconde quand l'étiquette reste dans le champ.

b) Flux Wi-Fi et capture. Le second sketch sert une image JPEG sur `/capture` et un flux MJPEG sur `/stream`. Reprenez la configuration des broches **CARTE_ESP32CAM** et la fonction `initCamera` du TP1.

```
// TP2 - partie 5b : flux MJPEG et capture JPEG (ESP32-CAM AI-Thinker)
#include "esp_camera.h"
#include <WiFi.h>
#include <WebServer.h>
// ... #define des broches CARTE_ESP32CAM et initCamera(...) du TP1 ...

const char *SSID = "EcoTri";          // à adapter
const char *MDP  = "motdepasse";
WebServer server(80);

void capture() {
  camera_fb_t *fb = esp_camera_fb_get();
  if (!fb) { server.send(500, "text/plain", "capture échouée"); return; }
  server.send_P(200, "image/jpeg", (const char *)fb->buf, fb->len);
  esp_camera_fb_return(fb);
}

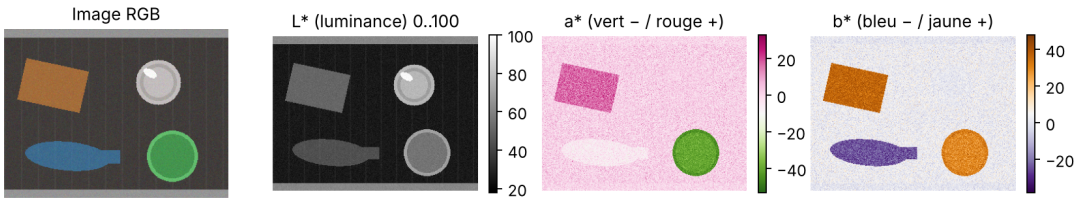
void stream() {
  WiFiClient c = server.client();
  c.print("HTTP/1.1 200 OK\r\nContent-Type: multipart/x-mixed-replace; boundary=frame
    \r\n\r\n");
  uint32_t n = 0, octets = 0, t0 = millis();
  while (c.connected()) {
    camera_fb_t *fb = esp_camera_fb_get();
    if (!fb) break;
    c.printf("--frame\r\nContent-Type: image/jpeg\r\nContent-Length: %u\r\n\r\n", fb
    ->len);
    c.write(fb->buf, fb->len); c.print("\r\n");
    octets += fb->len; n++;
    esp_camera_fb_return(fb);
    if (n % 50 == 0)
      Serial.printf("# flux : %.1f img/s, %.0f kbit/s\n", n * 1000.0 / (millis() - t0
    ),
                    octets * 8.0 / (millis() - t0));
  }
}

void setup() {
  Serial.begin(115200);
  initCamera(FRAMESIZE_QVGA, PIXFORMAT_JPEG);
  WiFi.begin(SSID, MDP);
  while (WiFi.status() != WL_CONNECTED) delay(300);
  Serial.printf("Ouvrir http://%s/stream ou /capture\n", WiFi.localIP().toString().
    c_str());
  server.on("/capture", capture);
  server.on("/stream", stream);
  server.begin();
}

void loop() { server.handleClient(); }
```

1. Ouvrez `/stream` dans un navigateur et relevez la cadence et le débit affichés sur le moniteur série, en QVGA puis en VGA. Comparez au débit théorique $D = M_{\text{JPEG}} \times f \times 8$.
2. **Couleur.** Le script suivant récupère une image sur `/capture` et mesure la couleur LAB d'un objet posé au centre. Mesurez 5 objets par classe et indiquez quelle composante (a^* ou b^*) sépare carton, verre et plastique.

```python
# lab_objet.py - sur le PC : couleur LAB moyenne de la zone centrale d'une capture
    ESP32-CAM
import sys, urllib.request, numpy as np, cv2
ip = sys.argv[1]
jpg = urllib.request.urlopen("http://%s/capture" % ip, timeout=5).read()
img = cv2.imdecode(np.frombuffer(jpg, np.uint8), cv2.IMREAD_COLOR)
h, w = img.shape[:2]
roi = img[h//2 - 30:h//2 + 30, w//2 - 30:w//2 + 30]
lab = cv2.cvtColor(roi, cv2.COLOR_BGR2LAB).reshape(-1, 3).astype(float)
L, a, b = lab[:, 0] * 100 / 255, lab[:, 1] - 128, lab[:, 2] - 128
print("L* %.1f  a* %.1f  b* %.1f  (taille JPEG %d o)" % (L.mean(), a.mean(), b.mean()
    , len(jpg)))
```



Rappel : sur b^* le carton est jaune et la bouteille bleue, sur a^* le verre est vert, et le métal reste neutre.

Question

Le sketch QR et le sketch flux ne peuvent pas tourner en même temps tels quels. Proposez une architecture logicielle qui fasse les deux (format de capture, partage du frame buffer, cœurs utilisés) et estimez la cadence de chaque usage à partir de vos mesures.

Partie 6 : Livrable du jalon J2 (15 min)

a) Tableau des mesures

Mesure	Valeur	Conditions
Cadence de la chaîne Portenta (FPS)		QVGA, ROI, IDE désactivé
Temps de traitement par image (ms)		`find_blobs` + descripteurs
Vitesse maximale du tapis suivie (cm/s)		partie 2, question 2
Erreurs de comptage sur 10 objets		
Exactitude des règles (IC 95 %)		$n =$
Rappel du métal		
Distance maximale de lecture du QR (cm)		QVGA
Cadence du flux MJPEG (img/s, kbit/s)		QVGA et VGA

b) Documentation du classifieur : arbre de règles (schéma), tableau des descripteurs par classe (moyenne et écart-type) et justification de chaque seuil.

c) Analyse des erreurs : trois objets mal classés, avec leur image (`img.save()`), leurs descripteurs et la raison de l'erreur. Pour chacun, dites si une règle supplémentaire pourrait le corriger sans en dégrader d'autres.

d) Message JSON : le format final émis par la Portenta, qui sera transmis par BLE à la lecture 3.

4 Grille d'évaluation du jalon J2

Critère	Attendu	Points
Fonctionnement	détection, suivi et comptage corrects ; JSON émis une fois par objet ; QR lu et flux affiché	5
Classifieur	règles justifiées par des mesures, seuils documentés	4
Évaluation	jeu de test indépendant, matrice de confusion, rappel par classe, intervalle de confiance	5
Analyse	erreurs expliquées, limites des règles argumentées, lien avec l'apprentissage	4
Qualité	code commenté, unités, conditions de mesure précisées	2
Total		**20**

5 Ressources

— Documentation OpenMV, module `image` : docs.openmv.io.

— Bibliothèque ESP32QRCodeReader (décodeur quirc) : github.com/alvarowolfx/ESP32QRCodeReader.

— Espressif, pilote `esp32-camera` et exemple *CameraWebServer* : github.com/espressif/esp32-camera.

— R. C. Gonzalez, R. E. Woods, *Digital Image Processing*, 4e éd., chapitres 10 (segmentation) et 11 (descripteurs).

— Harvard *Machine Learning Systems* (V. Janapa Reddi), chapitre sur la collecte et l'évaluation des données : mlsysbook.ai.